SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: MLA03
COURSE NAME: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO3: *Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

Code of Conduct:

I ___T Bhanu Teja/192472259___ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

T Bhanu Teja

QUESTIONS: 10 **Total Marks: 50**

Annexure A

CODING CHALLENGE **Duration: 2hrs**

1. **Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.**

Problem Statement

Reinforcement Learning (RL) is a machine learning technique where an intelligent **agent** learns by interacting with an **environment**. Instead of learning from labeled data, the agent observes the current state, selects an action, and receives a reward or penalty based on its action. Through repeated interactions, the agent learns an optimal strategy (policy) that maximizes long-term rewards.

Objectives

- Understand the basic Reinforcement Learning framework.
- Explain the interaction between the agent and the environment.
- Define state space, action space, and reward mechanism.
- Understand how an RL agent learns through trial and error.
- Explain the importance of cumulative rewards in decision-making.
- Demonstrate a simple RL implementation using Python.

Eligibility

Participants should have basic knowledge of:

- Python Programming
- Reinforcement Learning concepts
- Markov Decision Process (MDP)
- States, Actions, and Rewards
- Basic programming logic

Challenge Rules

- Clearly identify the **Agent** and **Environment**.
- Define the possible **States** and **Actions**.
- Design a simple **Reward Mechanism**.
- Calculate the **Cumulative Reward**.
- Continue learning until the goal state is reached.
- The objective is to maximize the total reward.

Programming Languages

The implementation can be done using:

- Python (Recommended)
- Java
- C++
- MATLAB

Python Libraries

No external libraries are required for this simple implementation.

Constraints

- The environment contains only three states: **Start**, **Middle**, and **Goal**.
- The available actions are **move** and **stay**.
- A positive reward is given for moving toward the goal.
- A negative reward is given for staying in the same state.
- The episode ends when the Goal state is reached.
- The agent should maximize the cumulative reward.

Python Program

```
# Simple Reinforcement Learning Framework

state = "Start"
```

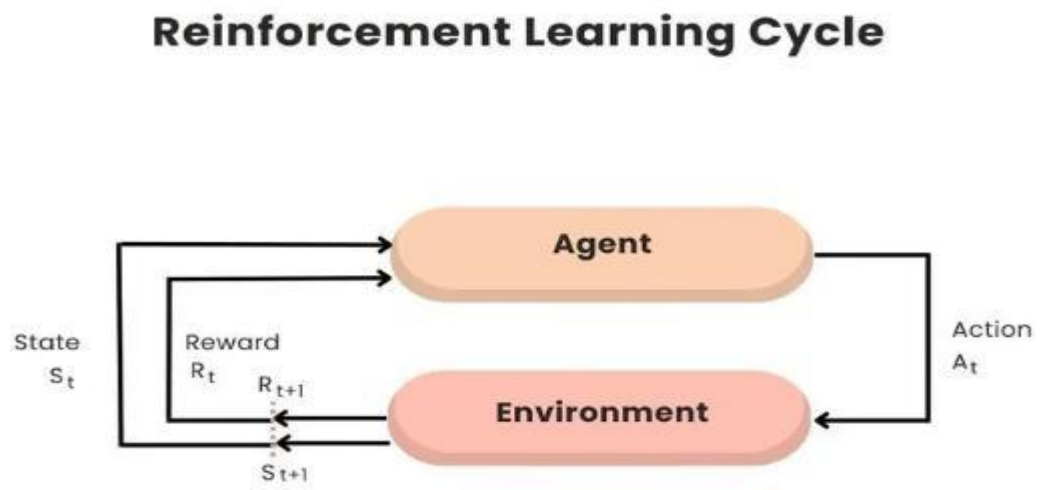
```
total_reward = 0
while state != "Goal":
    print("Current State:", state)
    action = input("Enter Action (move/stay): ")
    if action == "move":
        reward = 10
        total_reward += reward
        if state == "Start":
            state = "Middle"
        elif state == "Middle":
            state = "Goal"
    else:
        reward = -5
        total_reward += reward
    print("Reward:", reward)
    print("Total Reward:", total_reward)
    print()
print("Goal Reached!")
print("Final Reward:", total_reward)
```

Algorithm

1. Initialize the environment with the **Start** state.
2. Set the cumulative reward to **0**.
3. Read the action entered by the user.
4. If the action is **move**, move to the next state and give a positive reward.
5. If the action is **stay**, remain in the same state and give a negative reward.
6. Add the reward to the cumulative reward.
7. Repeat until the **Goal** state is reached.

8. Display the final cumulative reward.

```
Output                                                    Clear
Current State: Start
Enter Action (move/stay): move
Reward: 10
Total Reward: 10

Current State: Middle
Enter Action (move/stay): stay
Reward: -5
Total Reward: 5

Current State: Middle
Enter Action (move/stay): stay
Reward: -5
Total Reward: 0

Current State: Middle
Enter Action (move/stay): move
Reward: 10
Total Reward: 10

Goal Reached!
Final Reward: 10

=== Code Execution Successful ===
```



Expected Result

The RL agent successfully learns by interacting with the environment. By selecting appropriate actions and receiving rewards, it reaches the goal while maximizing the cumulative reward. This demonstrates the basic Reinforcement Learning framework consisting of **Agent**, **Environment**, **States**, **Actions**, **Rewards**, and **Cumulative Rewards**.

2 . Justify the exploration–exploitation trade-off in RL and compare strategies such as ε-greedy and softmax approaches in balancing learning efficiency.

Problem Statement

One of the biggest challenges in Reinforcement Learning (RL) is deciding whether an agent should **explore** new actions or **exploit** the actions that already produce good rewards. This is known as the **exploration–exploitation trade-off**. If the agent always explores, it may waste time trying poor actions. If it always exploits, it may miss better actions that have not yet been discovered.

The objective of this challenge is to understand the importance of balancing exploration and exploitation and compare two popular action-selection strategies: **ε-greedy** and **Softmax**. The solution should explain how these strategies improve learning efficiency and help the agent learn an optimal policy.

Objectives

The objectives of this challenge are:

- To understand the exploration–exploitation trade-off in RL.
- To explain why balancing exploration and exploitation is important.
- To compare ε-greedy and Softmax action-selection strategies.
- To understand how action-selection strategies improve learning efficiency.
- To demonstrate a simple implementation of the ε-greedy strategy.
- To analyze the advantages of different exploration methods.

Eligibility

Participants should have basic knowledge of:

- Python Programming
- Reinforcement Learning fundamentals
- States, Actions, Rewards
- Exploration and Exploitation concepts
- Probability and Random Selection

Challenge Rules

- Explain exploration and exploitation using suitable examples.
- Compare ε-greedy and Softmax strategies.
- Demonstrate how an agent selects actions.
- Maintain a balance between learning new actions and using the best-known action.
- Maximize cumulative rewards while improving learning efficiency.
- The solution should clearly justify why exploration is necessary during learning.

Programming Languages

The implementation can be done using:

- Python (Recommended)
- Java
- C++
- MATLAB

Python Libraries

No external libraries are required for the basic implementation.

Environment Setup

Step 1

Install Python 3.10 or above.

Step 2

Open any Python IDE such as:

- IDLE
- VS Code
- PyCharm
- Google Colab

Step 3

Create a new Python file named:

exploration_exploitation.py

Constraints

- ε value should be between **0 and 1**.
- The agent must choose one action at a time.
- Exploration selects a random action.
- Exploitation selects the action with the highest estimated reward.
- The objective is to maximize cumulative reward.
- Learning continues until the desired performance is achieved.

Python Program

```
import random
actions = ["A", "B", "C"]
rewards = [10, 18, 7]
epsilon = 0.2
if random.random() < epsilon:
    action = random.choice(actions)
    print("Exploration")
else:
    action = actions[rewards.index(max(rewards))]
    print("Exploitation")
print("Selected Action:", action)
print("Reward:", rewards[actions.index(action)])
```

Exploration–Exploitation Trade-off

Exploration

Exploration means trying **new or less frequently selected actions** to discover better rewards.

Examples:

- Trying a different route while navigating.
- Recommending a new movie to a user.
- Selecting a new strategy in a game.

Advantages:

- Discovers better actions.
- Prevents the agent from getting stuck.
- Improves long-term learning.

Disadvantage:

- May temporarily reduce rewards.

Exploitation

Exploitation means selecting the **best-known action** based on previous experience.

Examples:

- Choosing the shortest known path.
- Recommending the most popular product.
- Playing the strongest move in a game.

Advantages:

- Produces higher immediate rewards.
- Improves efficiency.
- Uses learned knowledge effectively.

Disadvantage:

- May ignore better undiscovered actions.

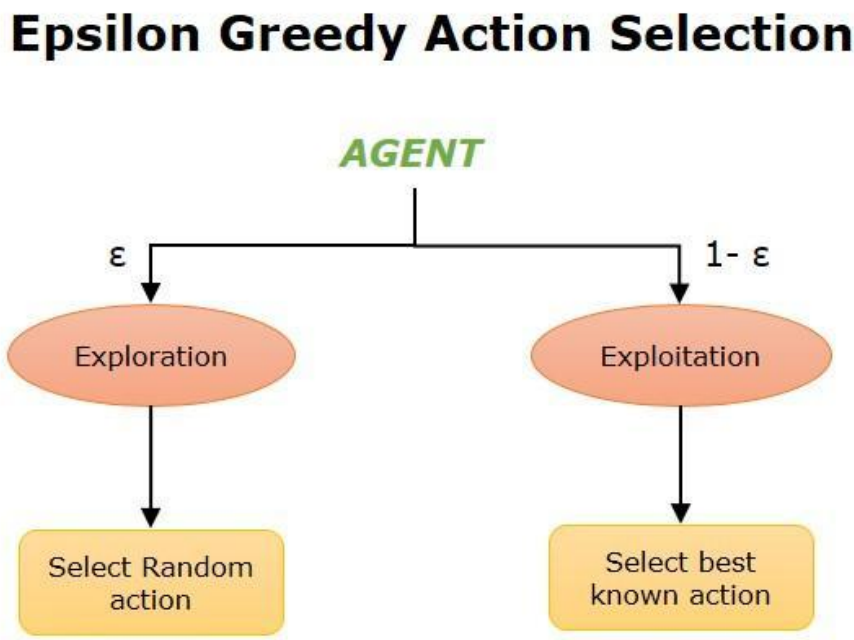
```
Output                                                    Clear
Exploitation
Selected Action: B
Reward: 18

=== Code Execution Successful ===
```

Expected Result

The RL agent successfully balances exploration and exploitation by selecting actions using ε-greedy or Softmax strategies. As learning progresses, the agent discovers better actions, increases cumulative rewards, and improves overall decision-making efficiency.

Conclusion

The exploration–exploitation trade-off is a fundamental concept in Reinforcement Learning. An agent must explore new actions to discover better strategies while exploiting previously learned knowledge to maximize rewards. The ε-greedy strategy offers a simple and effective balance, whereas the Softmax strategy provides more intelligent action selection by considering action probabilities. Properly balancing these strategies results in faster learning, better policy optimization, and improved long-term performance.



3. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies.

roblem Statement

In Reinforcement Learning, an agent learns by receiving rewards from the environment. However, if rewards are given only after reaching the final goal, learning may become slow because the agent receives very little feedback during training. **Reward Shaping** is a technique that provides additional intermediate rewards to guide the agent toward the desired behavior.

The objective of this challenge is to explain how Reward Shaping accelerates the learning process, improves learning efficiency, and prevents the agent from getting stuck in suboptimal (locally optimal but not best) policies.

Objectives

The objectives of this challenge are:

- To understand the concept of Reward Shaping.
- To explain how intermediate rewards improve learning speed.
- To understand the impact of Reward Shaping on policy optimization.
- To prevent the agent from learning suboptimal policies.
- To implement a simple Reward Shaping example using Python.
- To analyze how rewards influence agent behavior.

Eligibility

Participants should have basic knowledge of:

- Python Programming
- Reinforcement Learning fundamentals
- Reward Function
- Policy and Value Function
- Markov Decision Process (MDP)

Challenge Rules

- Define a reward function with intermediate rewards.
- Demonstrate how the agent receives rewards during learning.
- Show the difference between sparse rewards and shaped rewards.
- Prevent the agent from following inefficient paths.
- Maximize cumulative rewards while improving policy quality.
- Explain how Reward Shaping improves convergence speed.

Programming Languages

The implementation can be done using:

- Python (Recommended)
- Java
- C++
- MATLAB

Python Libraries

No external libraries are required for the basic implementation.

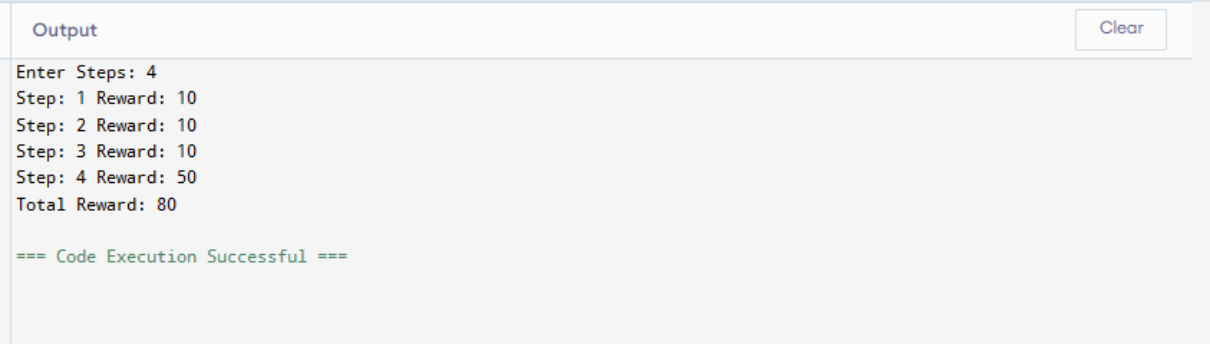
Constraints

- Rewards should encourage progress toward the goal.
- Intermediate rewards should not mislead the agent.
- The final goal should always provide the highest reward.
- The agent should avoid repeated unnecessary actions.
- The objective is to maximize cumulative reward.
- Reward values should remain consistent throughout training.

Python Program

Simple Reward Shaping Example

```
steps = int(input("Enter Steps: "))
total_reward = 0
for i in range(1, steps + 1):
    if i == steps:
        reward = 50      # Goal Reward
    else:
        reward = 10      # Intermediate Reward
    total_reward += reward
    print("Step:", i, "Reward:", reward)
print("Total Reward:", total_reward)
```

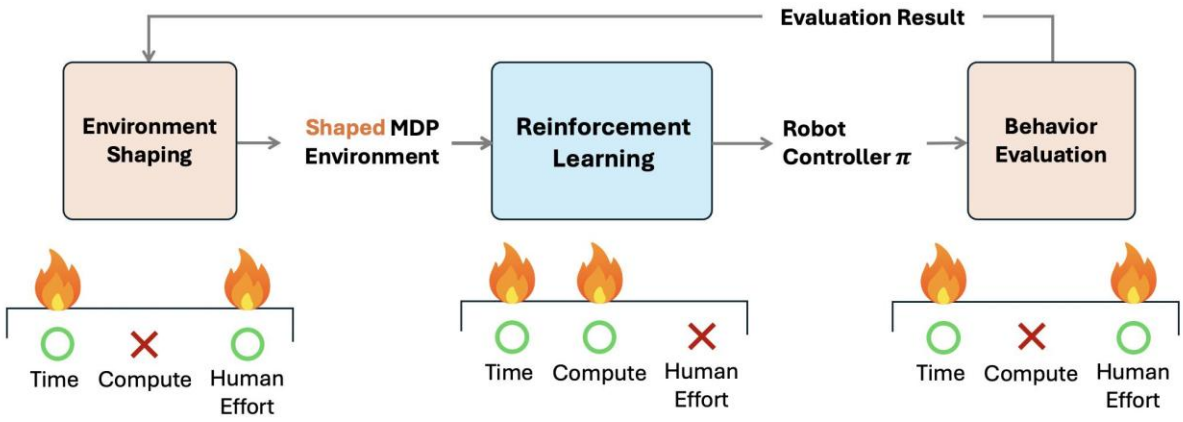


Expected Result

The RL agent successfully learns faster by receiving intermediate rewards during training. Reward Shaping improves learning efficiency, increases cumulative rewards, and helps the agent avoid suboptimal policies, resulting in better overall performance.

Conclusion

Reward Shaping is an effective technique in Reinforcement Learning that accelerates the learning process by providing additional feedback during training. Instead of waiting until the final goal is reached, the agent receives intermediate rewards that guide it toward better decisions. This reduces learning time, improves convergence, and prevents the agent from following suboptimal policies. Therefore, Reward Shaping plays a significant role in developing efficient and intelligent RL systems.



4. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

Problem Statement

Reinforcement Learning (RL) has become an important technique for developing intelligent robotic systems. Robots can learn to perform tasks such as navigation, object manipulation, warehouse automation, and industrial assembly without being explicitly programmed. However, applying RL in real-world robotics is challenging because robots interact with physical environments where mistakes can be costly or dangerous.

The objective of this challenge is to identify the major challenges of applying RL in robotics and analyze how safety, sample efficiency, and environment variability influence the learning performance of robotic agents.

Objectives

The objectives of this challenge are:

- To understand the application of RL in robotics.
- To identify the major challenges faced by RL-based robots.
- To analyze the importance of safety in robotic learning.
- To understand sample efficiency and its impact on training.
- To explain how changing environments affect robot performance.
- To demonstrate a simple RL-based robotic simulation.

Eligibility

Participants should have basic knowledge of:

- Python Programming
- Reinforcement Learning concepts
- Robotics fundamentals
- Sensors and Robot Navigation
- Artificial Intelligence basics

Challenge Rules

- Explain how RL is used in robotic systems.
- Identify challenges such as safety, sample efficiency, and environment variability.
- Analyze how these challenges affect learning performance.
- Demonstrate a simple robotic decision-making example.
- The robot should maximize task completion while avoiding unsafe actions.
- The solution should explain how RL improves robot performance over time.

Programming Languages

The implementation can be done using:

- Python (Recommended)
- C++
- Java
- MATLAB

Python Libraries

No external libraries are required for the basic implementation.

Environment Setup

Step 1

Install Python 3.10 or above.

Step 2

Open any Python IDE such as:

- IDLE
- VS Code
- PyCharm
- Google Colab

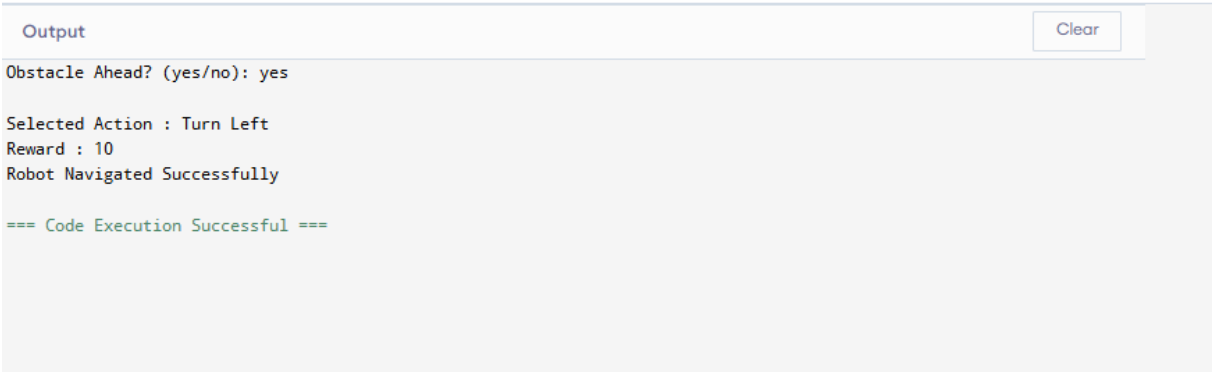
Step 3

Create a new Python file named:

robot_rl.py

Python Program

```
# Simple RL Robot Decision Program
obstacle = input("Obstacle Ahead? (yes/no): ")
if obstacle.lower() == "yes":
    action = "Turn Left"
    reward = 10
else:
    action = "Move Forward"
    reward = 20
print("\nSelected Action :", action)
print("Reward :", reward)
print("Robot Navigated Successfully")
```



Challenges of Applying RL in Robotics

1. Safety

Safety is the most important challenge in robotics. During the learning process, an RL agent may perform incorrect actions that can damage equipment or create unsafe situations.

Examples:

- Robot colliding with obstacles.
- Industrial robot damaging products.
- Delivery robot hitting pedestrians.

To improve safety:

- Use safe reward functions.
- Train robots in simulation before deployment.
- Apply safety constraints during learning.

2. Sample Efficiency

RL agents often require thousands of training episodes before learning an optimal policy.

Problems:

- Training consumes time.
- Physical robots experience wear and tear.
- High computational cost.

Solutions:

- Train in simulation first.

- Use transfer learning.
- Reuse previous experiences.

3. Environment Variability

Real-world environments are dynamic and unpredictable.

Examples:

- Moving obstacles.
- Lighting changes.
- Uneven surfaces.
- Weather conditions.

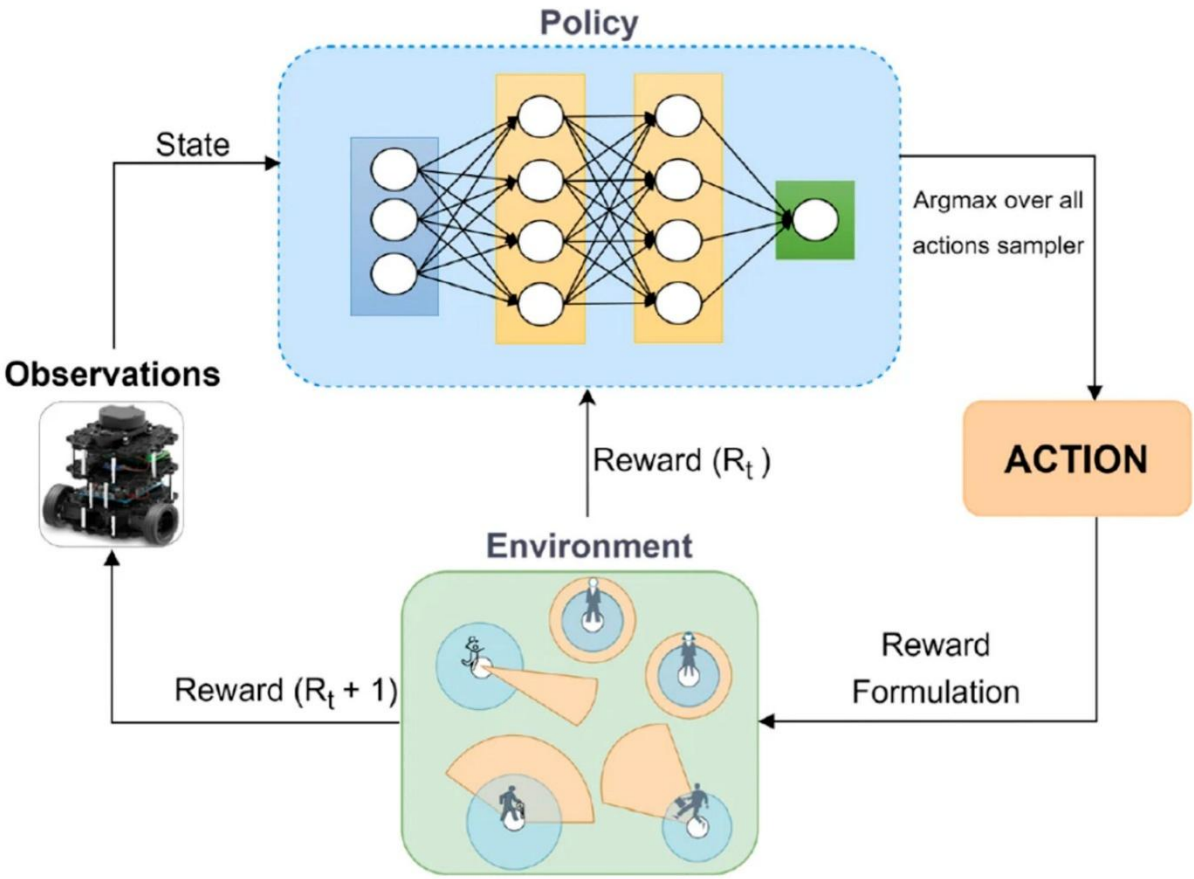
The RL agent must continuously adapt its policy to maintain good performance.

Expected Result

The RL-based robot successfully learns safe and efficient navigation strategies while adapting to different environments. By considering safety, improving sample efficiency, and handling environmental changes, the robot achieves better task completion with fewer errors and improved reliability.

Conclusion

Reinforcement Learning provides an intelligent approach for controlling robotic systems in dynamic environments. However, challenges such as safety, sample efficiency, and environment variability significantly influence learning performance. By designing appropriate reward functions, using simulation-based training, and continuously updating policies, RL agents can perform complex robotic tasks safely and efficiently. As research advances, RL is expected to play a major role in the future of autonomous robotics.


Policy
State
Argmax over all
actions sampler
Observations
ACTION
Reward (R_t)
Environment
Reward
Formulation
Reward (R_t + 1)

5. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations.

Problem Statement

Reinforcement Learning (RL) algorithms are broadly classified into Model-Free RL and Model-Based RL. Model-Free RL learns by interacting directly with the environment without knowing how the environment works. Model-Based RL first builds or uses a model of the environment and then plans the best actions before executing them.

The objective of this challenge is to compare Model-Free RL and Model-Based RL in dynamic environments. The comparison should explain their working principles, strengths, limitations, and effectiveness in solving real-world problems where the environment changes over time.

Objectives

The objectives of this challenge are:

- To understand Model-Free and Model-Based Reinforcement Learning.
- To compare their learning approaches.
- To evaluate their effectiveness in dynamic environments.
- To identify the strengths and limitations of both methods.
- To understand when each approach should be used.
- To implement a simple comparison using Python.

Eligibility

Participants should have basic knowledge of:

- Python Programming
- Reinforcement Learning concepts
- Markov Decision Process (MDP)
- Q-Learning
- Dynamic Programming

Challenge Rules

- Explain both Model-Free and Model-Based RL.
- Compare their working mechanisms.
- Identify strengths and limitations.
- Evaluate performance in dynamic environments.
- Demonstrate a simple comparison using Python.
- Clearly conclude which method is more suitable for different scenarios.

Programming Languages

The implementation can be done using:

- Python (Recommended)
- Java
- C++
- MATLAB

Python Libraries

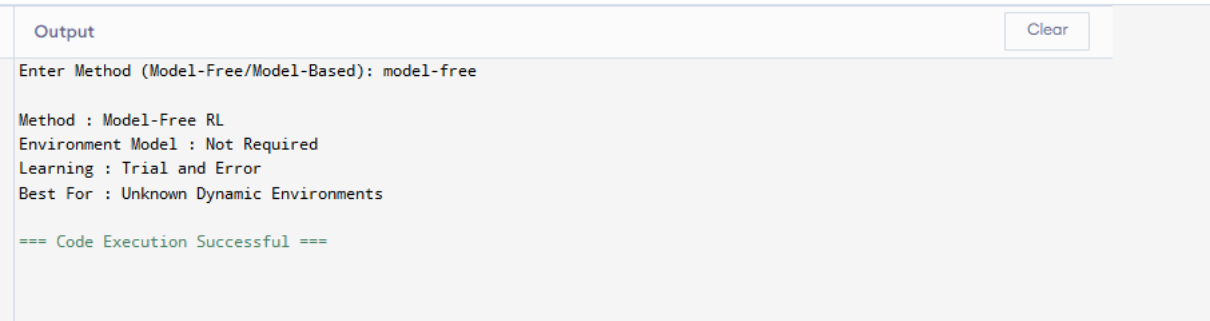
No external libraries are required for this simple comparison.

Constraints

- Only one RL approach should be selected.
- Dynamic environments may change during execution.
- Learning should improve with experience.
- The comparison should focus on adaptability and efficiency.
- Performance depends on the availability of an environment model.
- The objective is to achieve optimal decision-making.

Python Program

```
# Model-Free vs Model-Based RL

method = input("Enter Method (Model-Free/Model-Based): ")

if method.lower() == "model-free":

    print("\nMethod : Model-Free RL")

    print("Environment Model : Not Required")

    print("Learning : Trial and Error")

    print("Best For : Unknown Dynamic Environments"

elif method.lower() == "model-based":

    print("\nMethod : Model-Based RL")

    print("Environment Model : Required")

    print("Learning : Planning + Prediction")

    print("Best For : Known Environments")

else:

    print("Invalid Input")
```



Model-Free Reinforcement Learning

Model-Free RL does not require knowledge of the environment model. The agent learns by interacting directly with the environment through trial and error.

Examples

- Q-Learning

- SARSA
- Deep Q-Network (DQN)

Advantages

- Works well in unknown environments.
- Easy to implement.
- Suitable for complex real-world problems.
- No prior knowledge of the environment is required.

Limitations

- Requires many training episodes.
- Learning is slower.
- Less sample efficient.
- Exploration may take a long time.

Model-Based Reinforcement Learning

Model-Based RL uses a mathematical or learned model of the environment to predict future states before taking actions.

Examples

- Value Iteration
- Policy Iteration
- Dyna-Q

Advantages

- Learns faster.
- Requires fewer interactions.
- Better sample efficiency.
- Can plan future actions.

Limitations

- Requires an accurate environment model.

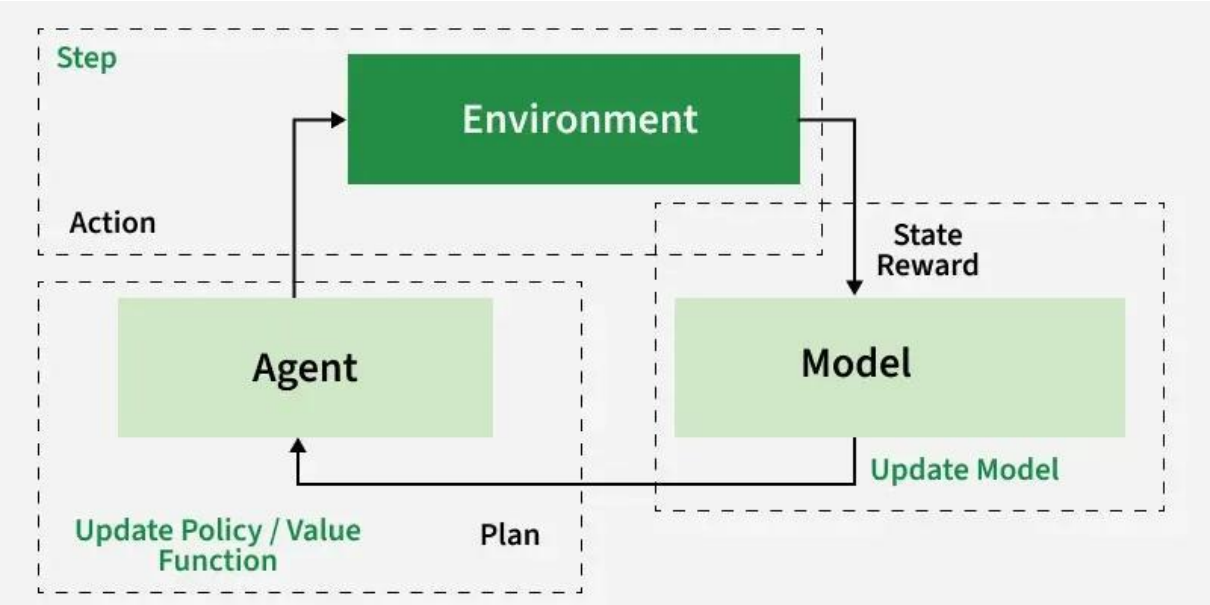
- Difficult to model complex environments.
- Less flexible when the environment changes frequently.

Expected Result

The comparison demonstrates that **Model-Free RL** is more adaptable in dynamic and uncertain environments because it continuously learns through interaction. **Model-Based RL** is more efficient when an accurate environment model is available, allowing faster planning and decision-making. Both methods have unique strengths, and the choice depends on the application.

Conclusion

Model-Free and Model-Based Reinforcement Learning are both powerful approaches for solving sequential decision-making problems. Model-Free RL is highly flexible and suitable for unknown or changing environments, while Model-Based RL provides faster learning and better planning when the environment model is known. Understanding their strengths and limitations helps in selecting the most appropriate method for applications such as robotics, autonomous vehicles, game playing, and industrial automation.



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately